CREDIT CARD FRAUD DETECTION

A Beginner-Friendly Machine Learning Project

Project type	Software-only / Machine Learning
Programming language	Python
Models	Logistic Regression and Random Forest (comparison)
Suggested interface	Streamlit web app

Project guide with problem statement, dataset, methodology, sample code, evaluation, testing, and future scope.

2. Abstract

Credit Card Fraud Detection is a machine learning project that tries to identify transactions that may be fraudulent. It learns patterns from historical transactions labelled as genuine or fraudulent and predicts a risk category for a new transaction. The goal is to help reviewers notice suspicious activity; the model does not prove that a transaction is fraud.

Fraud datasets often contain very few fraud cases compared with genuine transactions. This class imbalance makes accuracy alone misleading, so the project uses precision, recall, F1-score, a confusion matrix, and precision-recall analysis where possible.

3. Introduction

Credit cards make payments convenient, but stolen card details and unauthorised transactions can cause financial loss. Banks use multiple security checks to identify suspicious payments. Machine learning can support these checks by finding patterns that are less common in genuine transactions.

This student project uses a public, anonymised dataset and a simple model to demonstrate the workflow. It must not connect to real payment systems or process real card numbers.

4. Problem Statement

A large number of transactions may need to be checked quickly, while fraudulent transactions are rare. The aim is to build a software system that analyses transaction features and flags transactions that may be fraudulent for further review.

5. Objectives

- Understand a supervised machine learning classification problem.
- Prepare a labelled transaction dataset.
- Handle class imbalance and avoid relying on accuracy alone.

- Train and compare baseline classification models.
- Evaluate false positives and false negatives.
- Create a simple interface for demonstrating predictions on anonymised or synthetic feature values.

1. Project Scope

- Software-only project using Python.
- Use public, anonymised data or clearly labelled synthetic examples.
- Predict a risk label from the dataset's available features.
- No real bank account, payment gateway, or card network connection.
- Results are for education and should not be used for real financial decisions.

1. Existing System and Proposed System

Existing approach

Transaction monitoring may use rules such as unusual spending patterns, location changes, or known risk signals.

Rule-based checks are useful but can miss new patterns or flag genuine activity.

Proposed approach

A machine learning model learns from labelled historical examples and returns a predicted class or risk score. A high-risk prediction can be reviewed by a person. A student model is only a prototype and cannot replace bank-grade fraud controls.

2. Requirements

Hardware

- Laptop or desktop computer.
- At least 4 GB RAM recommended; larger datasets may need more memory.
- Internet access for installing libraries and downloading the dataset.

Software

- Python 3.10 or newer.
- VS Code, PyCharm, or Jupyter Notebook.
- Libraries: pandas, numpy, scikit-learn, joblib, streamlit, matplotlib (optional).
- A labelled CSV dataset with a fraud target column.

1. Dataset

A commonly used teaching dataset is the Credit Card Fraud Detection dataset hosted on Kaggle, based on European cardholder transactions from September 2013. It contains anonymised numerical features, Time, Amount, and a Class label (0 = genuine, 1 = fraud). Review the dataset page and terms before use. Its age and source mean that results may

not reflect current fraud patterns.

The sample code expects a CSV file named data/creditcard.csv and a target column named 'Class'. It assumes the feature columns are numeric. Do not use actual card numbers, CVV codes, PINs, passwords, or personal financial records.

Example fields

Field	Meaning
Time	Elapsed time measure in the dataset; not a real-world clock time
V1–V28	Anonymised numerical features provided by the dataset
Amount	Transaction amount in the dataset's unit
Class	0 = genuine transaction; 1 = fraudulent transaction

2. Methodology

- Collect a public anonymised dataset.
- Inspect columns, missing values, duplicates, and class distribution.
- Separate features from the target label.
- Split data into training and test sets using stratification to preserve the rare fraud class.
- Train a baseline Logistic Regression model with class weighting.
- Evaluate using precision, recall, F1-score, confusion matrix, and average precision.
- Optionally compare with a tree-based model using the same test split.
- Save the selected pipeline and build a demo interface.

1. System Architecture

```
Anonymised transaction features
|          v
Data validation and preprocessing
|             v Trained fraud classification
model             |             v Predicted class / risk
score             |             v Show result for
review            |             v Human review
(prototype only)
```

In a real financial system, the model would be only one component of a broader fraud-control process, with secure data handling, access controls, monitoring, and human or automated review policies.

2. Step-by-Step Implementation

Step 1: Create the folder structure

```
credit_card_fraud_detection /
■■■ data/
■   ■■■ creditcard.csv
■■■ models/
■   ■■■ fraud_model.joblib
■■■ train_model.py
■■■ app.py
■■■ requirements.txt
■■■ README.md
```

Step 2: Install packages

```
python -m venv .venv # Activate the environment, then run: pip install
pandas numpy scikit-learn joblib streamlit
```

Step 3: Download and prepare data

Place the public dataset at data/creditcard.csv. Check that 'Class' contains 0 and 1. The code below removes rows with missing values for simplicity; for a real report, explain your cleaning choices and check whether dropping rows changes the class balance.

Step 4: Train and evaluate

```
# train_model.py from pathlib
import Path import joblib import
numpy as np import pandas as
pd
from sklearn.model_selection import train_test_split from
sklearn.pipeline import Pipeline from sklearn.preprocessing import
StandardScaler from sklearn.linear_model import LogisticRegression
from sklearn.metrics import ( classification_report,
confusion_matrix, average_precision_score, roc_auc_score )

df = pd.read_csv ("data/creditcard.csv") df =
df.dropna() y = df["Class"]. astype(int) X =
df.drop(columns=["Class"])

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2,
random_state=42, stratify= y )

pipeline = Pipeline([ ("scaler", StandardScaler()), ("model",
LogisticRegression( class_weight="balanced", max_iter=1000,
random_state=42 )) ])

pipeline.fit(X_train, y_train) pred =
pipeline.predict(X_test) prob =
pipeline.predict_proba(X_test)[:, 1]

print("Classification report:")
print(classification_report(y_test, pred, zero_division=0)) print("Confusion matrix (rows=actual,
columns=predicted; labels 0,1):") print(confusion_matrix(y_test, pred, labels=[ 0, 1]))
print("Average precision:", round(average_precision_score(y_test, prob), 4)) print("ROC-AUC:",
round(roc_auc_score(y_test, prob), 4))

Path("models"). mkdir(exist_ok=True) joblib.dump({"pipeline": pipeline, "columns":
X.columns.tolist()}, "models/fraud_model.joblib") print("Saved model to models/
fraud_model.joblib")
```

This is a teaching baseline. Class weighting can help the model pay attention to rare fraud cases, but it does not guarantee good performance. Choose a decision threshold based on the evaluation and the intended review workload.

Do not report results until you run the code.

Step 5: Create a demo interface

```
# app.py import joblib import
pandas as pd import
streamlit as st

saved = joblib.load("models/fraud_model.joblib") pipeline =
saved["pipeline"] columns = saved["columns"]

st.title("Credit Card Fraud Detection") st.write( "Educational demo only. Enter
anonymised feature values from "
  "the dataset; do not enter real card details." )

values = {} for col in columns: default =
0.0 values[col] = st.number_input(
    col, value=float(default), format="%.6f" )

if st.button("Check transaction"):
  row = pd.DataFrame ([[values[c] for c in columns]], columns= columns) risk =
float(pipeline.predict_proba(row)[ 0, 1]) st.write(f"Model fraud-risk score: {risk:.4f}") if risk
>= 0.5:
    st.warning("Flagged as potentially fraudulent for review.") else:
    st.success("Not flagged by this model; this does not prove it is safe.") st.caption( "Never use this
prototype to approve or reject real transactions."
  )
```

The app uses zero as a demonstration default for every feature. This is not a realistic transaction and should not be presented as a meaningful prediction. For a final demo, select

a few anonymised test rows and show the model's outputs alongside their known labels.

Step 6: Run the project

```
python train_model.py  streamlit run
app.py
```

Open the local address printed by Streamlit, commonly http://localhost:8501.

3. Evaluation Metrics

- Precision: among transactions flagged as fraud, how many are actually fraud.
- Recall: among all fraud cases, how many the model identifies.
- F1-score: combines precision and recall.
- Confusion matrix: shows true positives, false positives, true negatives, and false negatives.
- Average precision / precision-recall curve: useful for imbalanced fraud datasets.
- ROC-AUC: measures ranking ability across thresholds, but should not be used alone for rare fraud.

Accuracy can be misleading when genuine transactions vastly outnumber fraud. Report the class distribution, evaluation method, decision threshold, andall importantmetrics.

1. Testing Plan

Test	Action	Expected behaviour
Genuine sample	Use an anonymised test row labelled 0	Record prediction and compare with label
Fraud sample	Use an anonymised test row labelled 1	Record prediction and compare with label
Missing values	Check data before training	Cleaning or validation handles missing values
Rare class	Inspect train/test class counts	Both sets contain fraud cases due to stratification
Invalid schema	Remove or rename a feature column	Training fails clearly or schema check identifies issu

2. Expected Outcome

The project should load anonymised transaction data, train a model, produce evaluation metrics, save the model, and provide a demo interface showing a risk score. Actual results depend on the dataset, model, threshold, and split.

3. Advantages

- Demonstrates a real-world machine learning problem.
- Shows how to evaluate models when fraud cases are rare.
- Uses common open-source Python libraries.
- Can be extended with additional anonymised transaction features.

1. Limitations

- Historical fraud patterns may not match current fraud methods.
- An anonymised dataset may omit important real-world signals.
- False positives can inconvenience genuine cardholders; false negatives can miss fraud.
- A model trained on one dataset may not work well on another bank or region.
- The demo is not secure or validated for real payment use.

1. Future Enhancements

- Compare Logistic Regression, Random Forest, and gradient-boosted models.
- Tune the decision threshold using precision-recall trade-offs.
- Use time-based validation to better reflect future transactions.
- Add drift monitoring and retraining plans.
- Add explainability for why a transaction was flagged.
- Evaluate fairness, privacy, access control, and secure deployment before any real integration.

1. Conclusion

Credit Card Fraud Detection is a useful machine learning project for learning classification, imbalanced datasets, evaluation metrics, and simple application development. It can flag suspicious examples for demonstration, but a student prototype must never be used to make real payment decisions.

2. References and Learning Resources

- Scikit-learn documentation: https://scikit-learn.org/
- LogisticRegression documentation:https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- Model evaluation documentation: https://scikit-learn.org/stable/modules/model_evaluation.html
- Streamlit documentation: https://docs.streamlit.io/
- Kaggle Credit Card Fraud Detection dataset: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud.